Who’s that bot? A Pokemon Showdown Bot Through Reinforcement Learning

Abstract

...

Related Works

Winning at Pokémon Random Battles Using Reinforcement Learning

Wang (2024) presents a bot designed to play random battles in Generation 4. The agent is modeled as a Partially Observable Markov Decision Process (POMDP), trained using Proximal Policy Optimization (PPO), and further improved at inference time through Monte Carlo Tree Search (MCTS).

Introduction

Problem Statement

Since Deep Blue defeated Kasparov in 1997, the use of AI in turn-based games has been extensively explored. Games like chess and Go now have AIs capable of defeating the best human players, Stockfish in chess and AlphaGo in Go, yet there is still no bot capable of consistently beating the top Pokémon VGC players to this day. And it's not that Pokémon is necessarily harder than chess, but two things are certainly true. First, it has a larger number of possible game states than chess. Second, it's much harder to tell whether an AI is winning or losing at any given moment, unlike chess, where having more pieces and more paths to checkmate are reliable indicators of advantage. In Pokémon, you can be down two Pokémon and still be winning.

Background

Pokemon VGC

Pokémon VGC is Nintendo's official format for Pokémon battles. It consists of 2-on-2 matches, where each player brings a team of 6 Pokémon, but at the start of each battle they select only 4 of them, and of those 4 only 2 can be on the field at the same time. Each Pokémon can hold an equipped item, and throughout the battle players take turns simultaneously choosing an action for each of their two active Pokémon, either attacking or switching in another member of the team. The winner is whoever leaves the opponent with no Pokémon remaining, or whoever has more Pokémon standing when the battle timer runs out.

Pokemon Showdown

Pokémon Showdown is a free, open-source Pokémon battle simulator that runs directly in the browser. Unlike the official video games, where players need to invest hundreds of hours obtaining and training their Pokémon, Showdown allows anyone to build a competitive team

in a matter of minutes and start battling right away. This makes it the perfect tool for the competitive community to practice, test strategies, and prepare for official tournaments.And in this case train our IA.

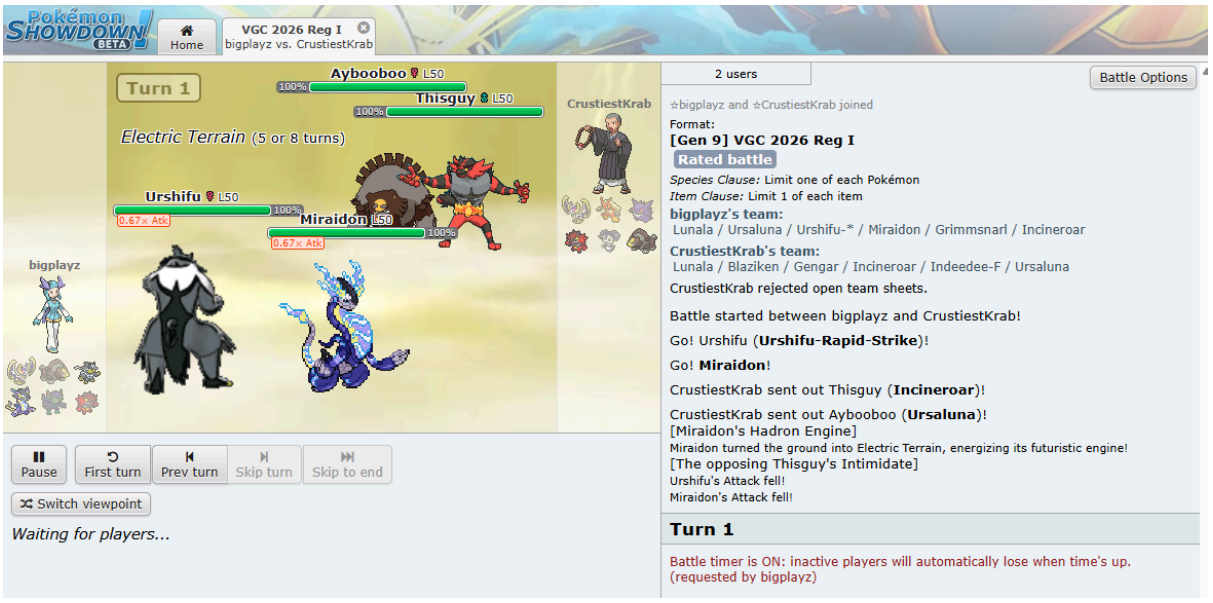


Figure 1.1: A Gen 9 VGC 2026 regulation I match in Pokemon Showdown.

Environment

Environment Description

The agent will be trained in a local Pokémon Showdown server running on Node.js, interacting with it through the Python library poke-env. Battles will take place in the VGC 2026 Regulation M-A, and the agent will use a fixed team built by a human, which will remain the same throughout the entire training process. A battle ends when all Pokémon on one side have fainted.

State Space

At the start of each turn, the agent receives a complete state of the battle state and of its team. It knows the current HP of all its Pokémon, which Pokémon are still available in reserve, the moves, their held items, and any active stat boosts. On the opponent's side, the agent has full knowledge of the rival's current HP, the Pokémon visible on the field, their used moves, and any active stat boosts, the agent also receives information about the general battle state, including the current weather and terrain, any field effects such as Tailwind or Trick Room, which Pokémon on either side have fainted, and the current turn number.

State Space:
Own side**:**

- Current HP of all its Pokémon

- Which Pokémon are still available in reserve
- Available moves
- Held items
- Active stat boosts

Opponent's side:
- Current HP of all opponent's Pokémon
- Pokémon visible on the field
- Moves that have been used
- Active stat boosts

General battle state:
- Current weather and terrain
- Active field effects
- Current turn number

Action Space

Each turn, the agent chooses an action for each of its two active Pokémon. For each one, it can either use one of its available moves or switch it out for a Pokémon from the reserve. When using a move, the agent must also specify a target if the move requires one, either one of the two opposing Pokémon or its own ally, whereas moves that hit all Pokémon on the field do not require a target selection. Additionally, once per battle the agent may Terastallize one of its active Pokémon, which changes its type and can be combined with a move action on the same turn.

Action Space:
- Use a move
- Switch out
- Terastallize

Reward Function

The reward function is based on Wang (2024). The agent receives a reward of +1 upon winning a battle, -1 upon losing, and 0 on every other turn. This reward structure allows the agent to learn strategies that play out over multiple turns, such as moves with effects that last several turns or setups that only pay off later in the battle, something that would not be possible if the reward function assigned value to individual turn outcomes.

Methods

Training

The agent will play against itself to train the model. We will save the SimpleHeuristicsPlayer, a pokemon bot, for validation. According to Wang (2024), the bot is easy enough for the agent to beat, but difficult enough for it to learn from. This balance is precisely why RandomPlayer and MaxBasePowerPlayer were not used to train the model, RandomPlayer selects actions at random, meaning the agent would learn nothing meaningful from it, while the MaxBasePowerPlayer always picks the highest base power move, which could teach the agent the bad habit of always assuming that the strongest move is the best choice.

Validation

The metrics used to measure the model's success will be based on Wang (2024). We will use win rate, against SimpleHeuristicsPlayer, as the primary success metric, measured every 20,000 steps, which according to the paper corresponds to approximately 200 battles against the bot.

Agent 1

Architecture

The agent uses a deep neural network with 12 inputs, followed by 3 fully connected layers of 256, 128, and 64 neurons respectively, with ReLU activation, ending in an output layer with linear activation. The output produces 3 Q-values, one for each strategy the agent can choose(A random move, The strongest attack or A simple heuristic move). The network uses the Adam optimizer with a learning rate of 0.001 and mean squared error as the loss function. Softmax is not used because we care about Q-values and not the probability of each action, and mean squared error is used instead of cross-entropy because the latter is for classification, while mean squared error better penalizes large errors during training.

Training

At each turn, the agent receives the current battle state: the health points of the 4 active Pokémon and a binary value (0 or 1) indicating the availability of each Pokémon's moves. With this state, the agent calls the model to return an action.

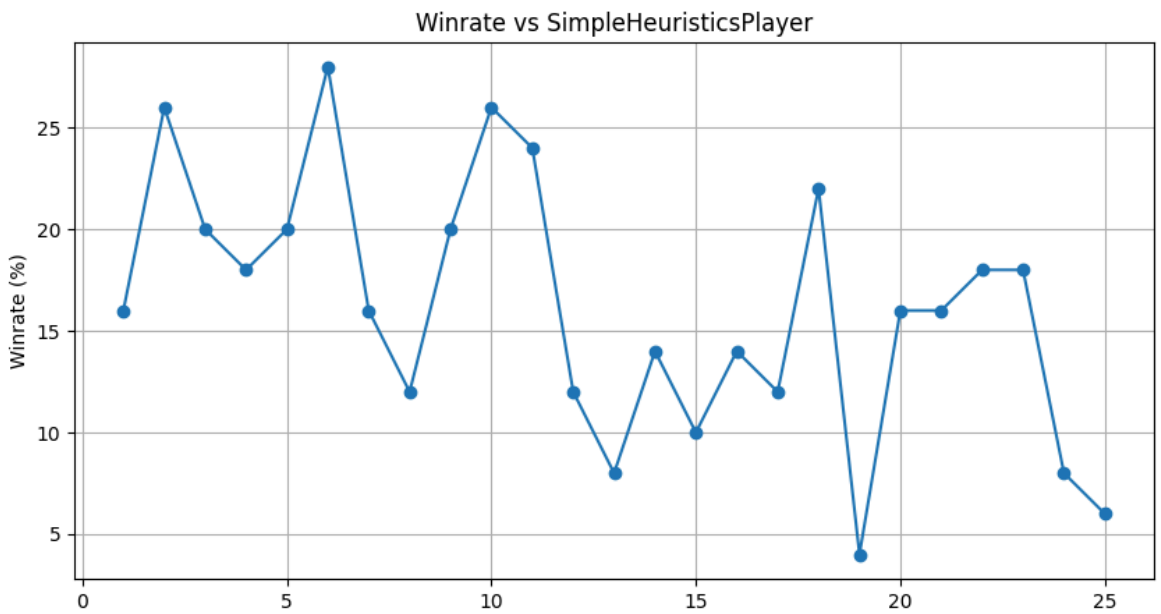
Starting from the second turn, the agent stores in memory the previous battle state, the previous action taken, the reward received, the current state and the current action. The agent requires a minimum of 64 stored experiences before it can begin learning.

During training, the agent samples 64 random experiences from memory and uses the Bellman equation to calculate the target Q-value for each experience. This target Q-value is then used to adjust the network's parameters via backpropagation.

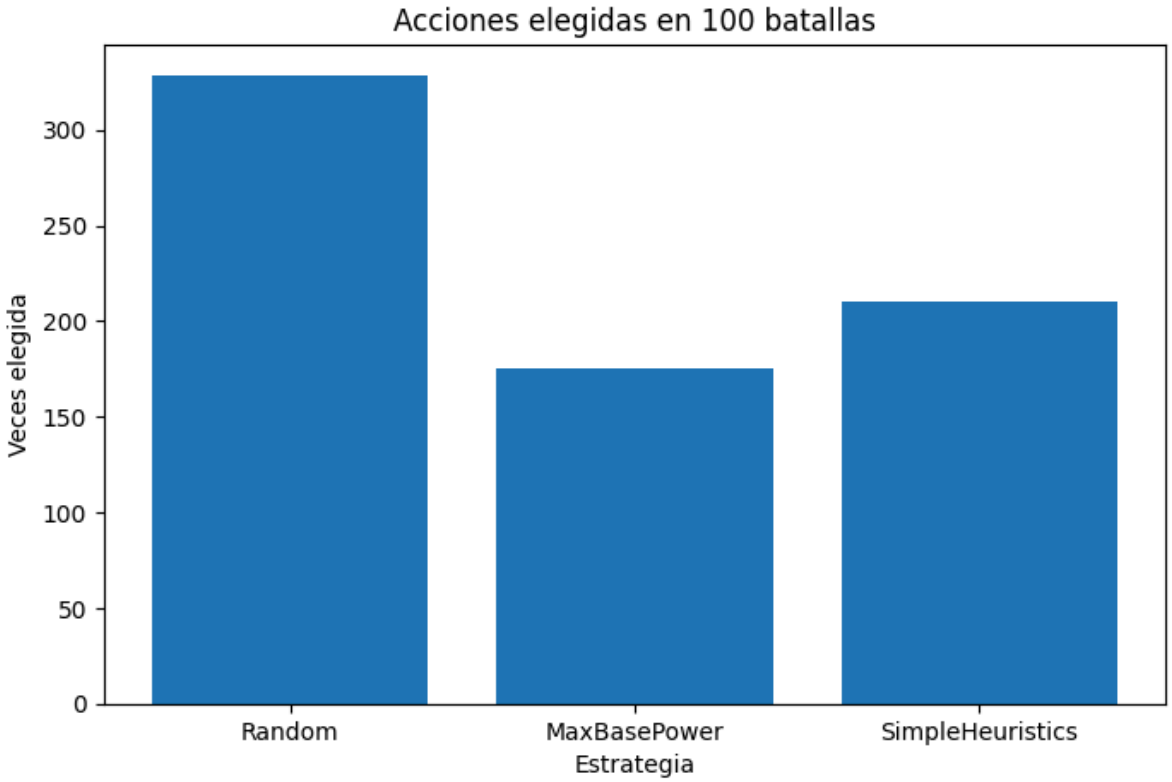
When choosing an action, the agent uses an epsilon-greedy policy. Epsilon starts at 1.0 and decays over time. At each turn, a random value between 0 and 1 is generated, if it is less than epsilon, the agent explores by choosing a random action; otherwise, its action is based on what it has learned by choosing the action with the highest Q-value.

$$Q(s_t, a_t) = r_t + \boxed{\gamma Q(s_{t+1}, a_{t+1})}$$

Results



The graph shows the agent's win rate against SimpleHeuristicsPlayer over 25 validation checkpoints, each taken after 200 training battles, for a total of 5,000 self-play battles. Training took approximately 6 hours. The win rate oscillates between 4% and 28% throughout training with no clear upward trend, and ends near 6% at the final checkpoint.



This graph shows that the agent chose random moves most of the time, when in fact that strategy should only be used occasionally to avoid being predictable. The agent's best option would have been simple heuristics, while max base power should be its second choice when an aggressive play is needed. Perhaps too few battles were played for the agent to learn that.

Conclusions and future steps

In conclusion, this model does not appear to be learning to make optimal decisions. Currently the agent can only choose between 3 strategies: random move, max base power move, and simple heuristics move. One would expect the agent to learn to choose the simple heuristics strategy most of the time, which would yield at least a 50% win rate.

Several factors may explain this failure. First, epsilon decayed too quickly, by the time the first validation checkpoint was reached, epsilon had already dropped to its minimum value, meaning the agent stopped exploring before learning anything meaningful. For the next attempt, epsilon will decay once per battle instead of once per turn.

Second, the agent learns from 64 random experiences sampled from its entire history. In Pokémon, early-game decisions have a greater impact on the outcome than late-game ones, so sampling randomly may not be the best idea for learning. A better approach would be to learn from the last 5 to 10 turns instead, and increasing gamma from 0.99 to 1.0 could also help the model better value long-term outcomes.

Third, the action space may be too limited. Initially the idea was to allow the agent to choose specific moves, but due to implementation difficulties and training tim, it was simplified to only 3 strategies. It now appears that significantly more training time will be needed, and allowing the agent to choose moves more specifically may be necessary to achieve competitive performance.

Finally, we will increase the state vector, so the agent can use more knowledge of the environment to make its predictions.

Agent 2

Architecture

The architecture of this model is identical to the previous one, with the same number of neurons, layers, activation functions, inputs, and outputs. The only changes made are to the training process and how often the model parameters are updated.

Training

The first change was to slow down the epsilon decay rate. According to Perkins et al. (2025), a decay rate that is too passive can increase the temporal complexity of the algorithm yet a very aggressive decay can prevent it from finding the optimal policy. For this reason, epsilon decay was reduced from 0.995 to 0.9975, allowing the agent to explore more before converging.

The second change was to increase gamma. According to Mnih et al. (2013) in Playing Atari with Deep Reinforcement Learning, future rewards are discounted by a factor of gamma for each step taken. This means that strategies that require more time to see their benefits need a higher gamma so that the reward signal is not lost. Based on Wang (2024), gamma was increased from 0.99 to 0.9999.

The third change, motivated by Fedus et al. (2020) in Revisiting Fundamentals of Experience Replay, addresses the replay buffer size. The abstract discusses how buffer size can be beneficial for some algorithms but not others, and highlights that controlling the replay ratio is important in RL algorithms. For this reason, instead of training on a fixed batch of 64 random experiences per turn, the model now learns from all the turns of the current battle at the end of each game, and replacing per-turn updates with per-battle updates.

Results

[Se está entrenado todavía el modelo]

Results

Analysis and comparison

[Se está entrenado todavía el modelo]

Conclusions

Resume

Limitations

Future Possible Work

References

[1] Wang, J. (2024). Winning at Pokémon random battles using reinforcement learning (Master's thesis). Massachusetts Institute of Technology.